Admin Web Technical Documentation

Legacy and newer admin surfaces and controls.

Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository: rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @ 3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Legacy and newer admin surfaces and controls. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

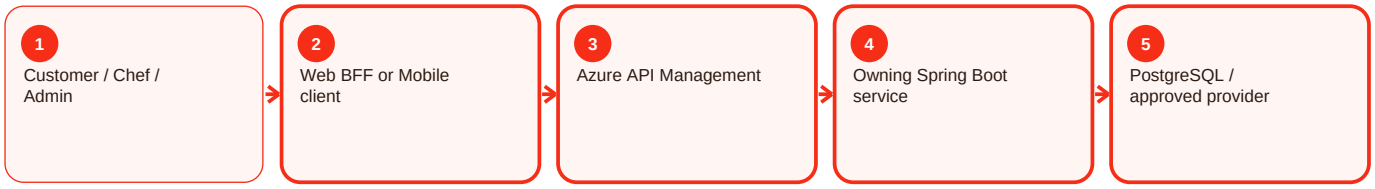
A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

Admin surface boundary - Overview and responsibility

Current implementation

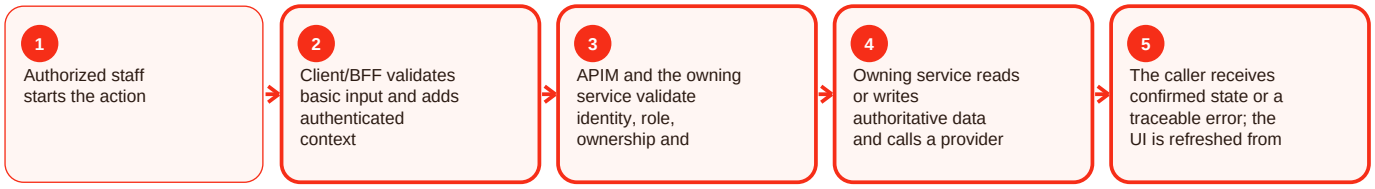
Summary

Admin surface boundary is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin surface boundary, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin surface boundary - Functional behavior and data

Current implementation

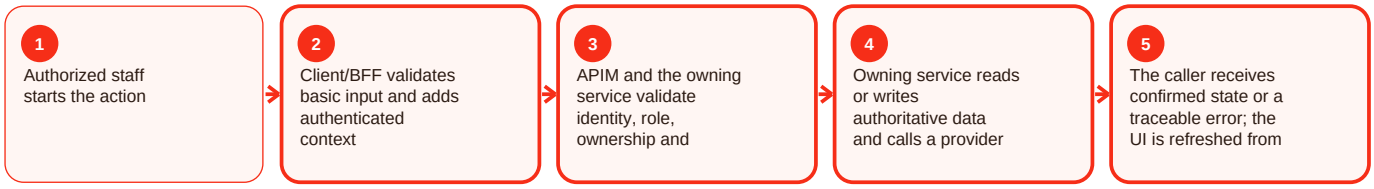
Summary

Admin surface boundary is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin surface boundary, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin surface boundary - Operations, errors and deployment

Current implementation

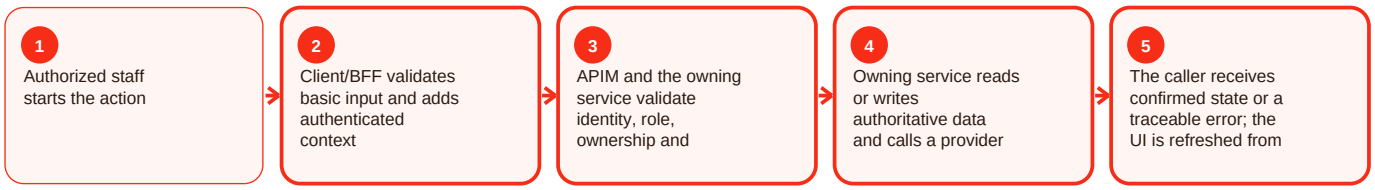
Summary

Admin surface boundary is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin surface boundary, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Legacy apps / admin - Overview and responsibility

Legacy / reference

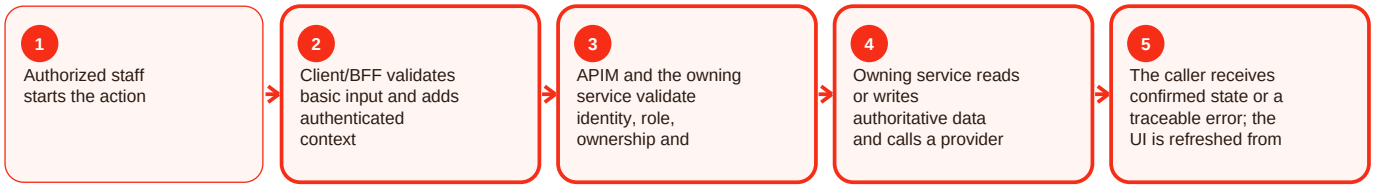
Summary

Legacy apps / admin is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For legacy apps/admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Useful for history and compatibility; it is not treated as the preferred current runtime. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Legacy apps / admin - Functional behavior and data

Legacy / reference

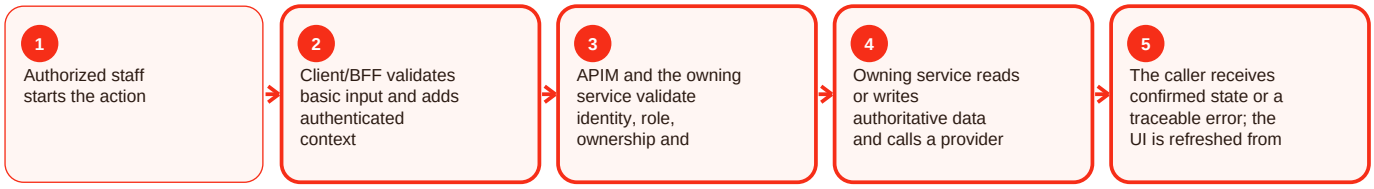
Summary

Legacy apps / admin is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For legacy apps/admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Useful for history and compatibility; it is not treated as the preferred current runtime. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Legacy apps / admin - Operations, errors and deployment

Legacy / reference

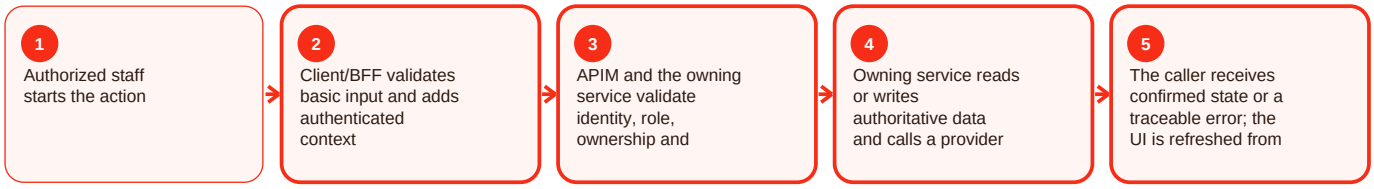
Summary

Legacy apps / admin is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For legacy apps/admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Useful for history and compatibility; it is not treated as the preferred current runtime. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

New admin shell - Overview and responsibility

Current implementation

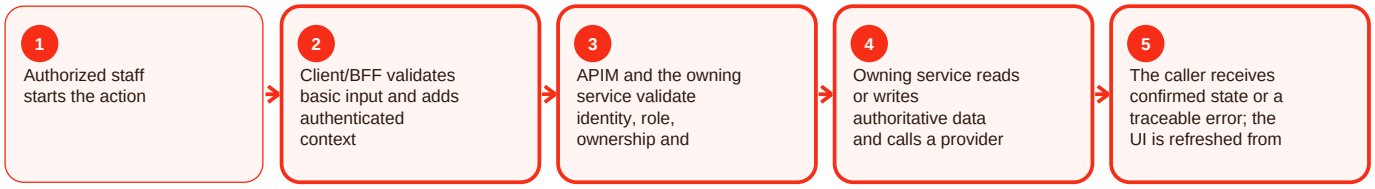
Summary

New admin shell is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For new admin shell, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

New admin shell - Functional behavior and data

Current implementation

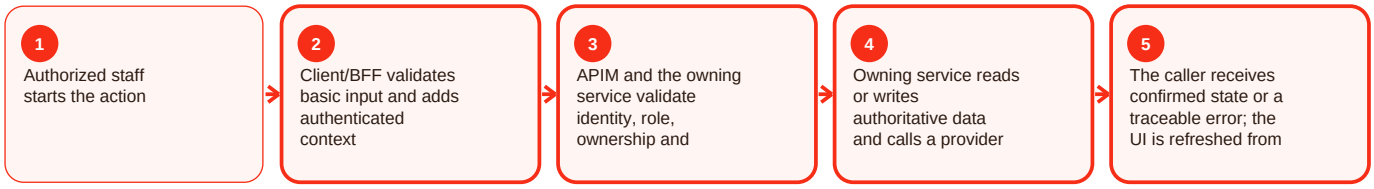
Summary

New admin shell is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For new admin shell, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

New admin shell - Operations, errors and deployment

Current implementation

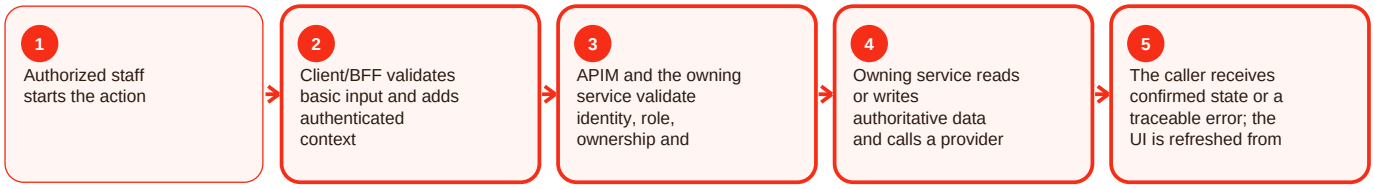
Summary

New admin shell is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For new admin shell, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim admin product - Overview and responsibility

Current implementation

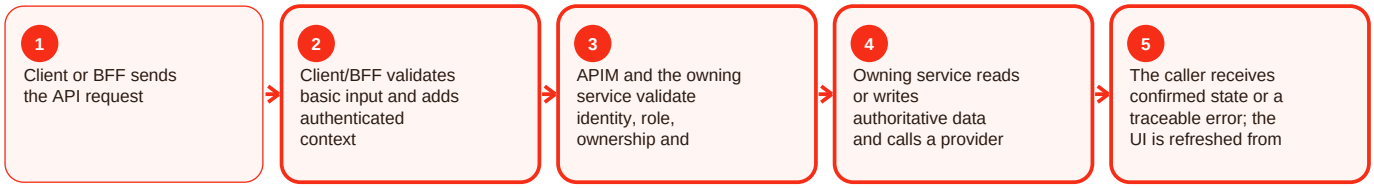
Summary

Apim admin product is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM admin product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim admin product - Functional behavior and data

Current implementation

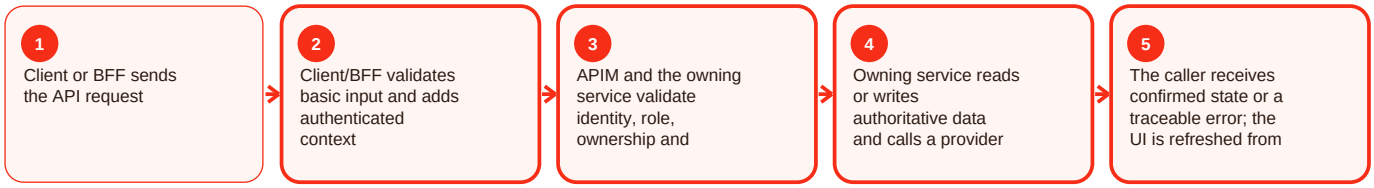
Summary

Apim admin product is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM admin product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Apim admin product - Operations, errors and deployment

Current implementation

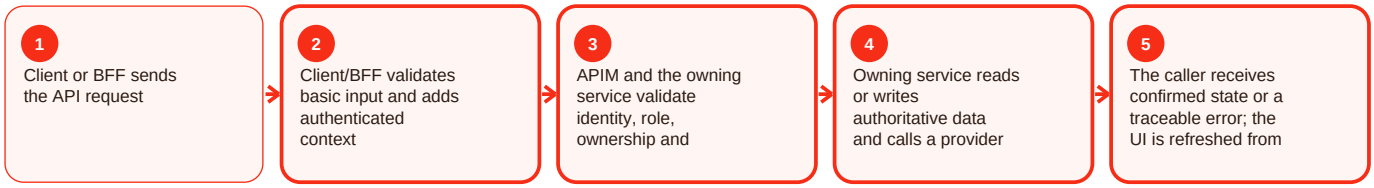
Summary

Apim admin product is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For APIM admin product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscription protection - Overview and responsibility

Current implementation

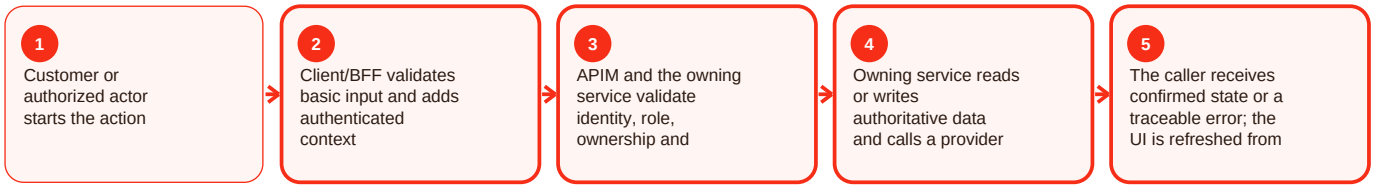
Summary

Subscription protection is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscription protection, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscription protection - Functional behavior and data

Current implementation

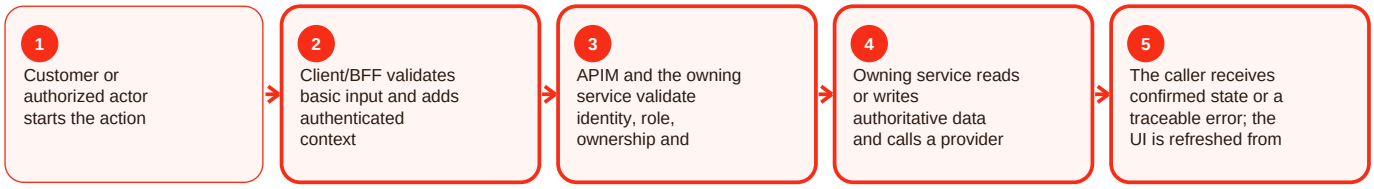
Summary

Subscription protection is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscription protection, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscription protection - Operations, errors and deployment

Current implementation

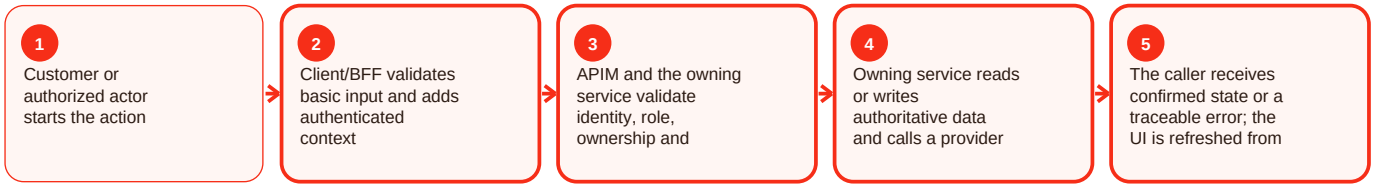
Summary

Subscription protection is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscription protection, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Role policy - Overview and responsibility

Current implementation

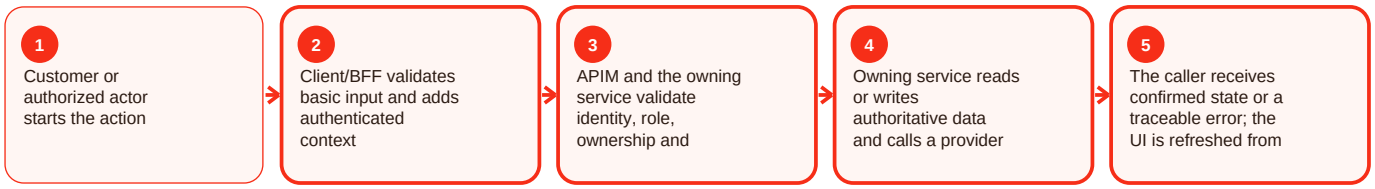
Summary

Role policy is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For role policy, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md; services/user-chef-service/README.md; services/order-service/README.md; apps/customer-web-next/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Role policy - Functional behavior and data

Current implementation

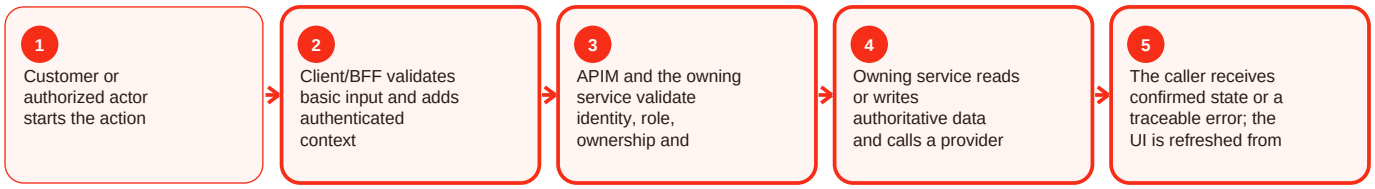
Summary

Role policy is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For role policy, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md; services/user-chef-service/README.md; services/order-service/README.md; apps/customer-web-next/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Role policy - Operations, errors and deployment

Current implementation

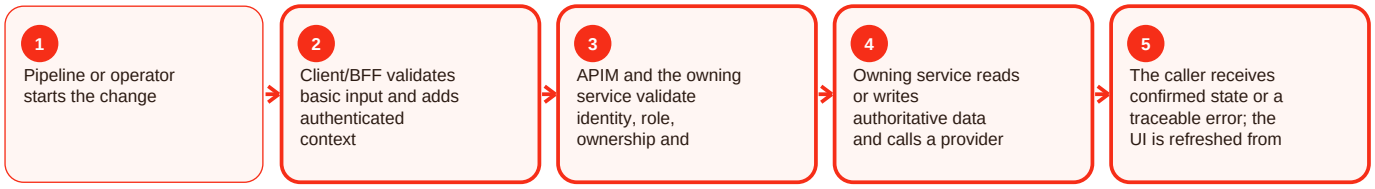
Summary

Role policy is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For role policy, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef review - Overview and responsibility

Current implementation

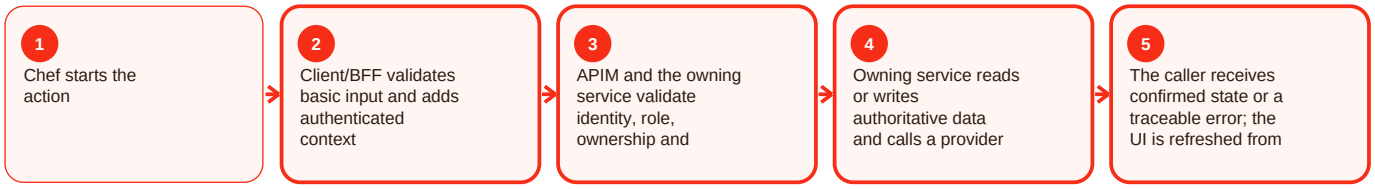
Summary

Chef review is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef review, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef review - Functional behavior and data

Current implementation

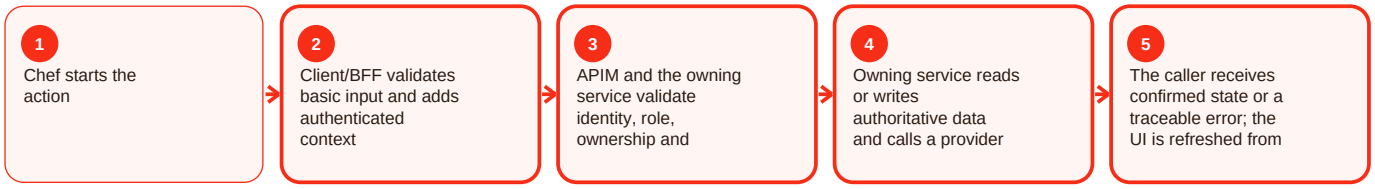
Summary

Chef review is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef review, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef review - Operations, errors and deployment

Current implementation

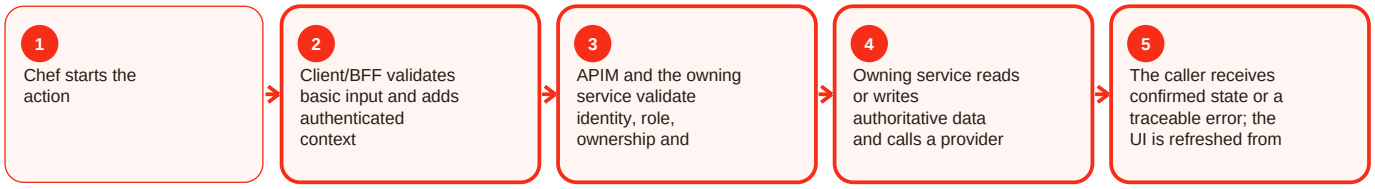
Summary

Chef review is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef review, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef verification - Overview and responsibility

Current implementation

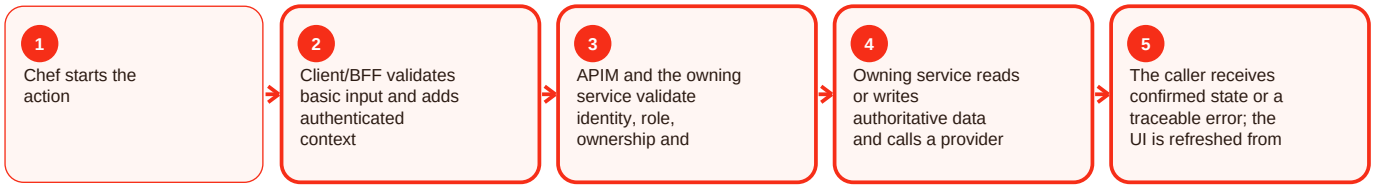
Summary

Chef verification is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef verification, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef verification - Functional behavior and data

Current implementation

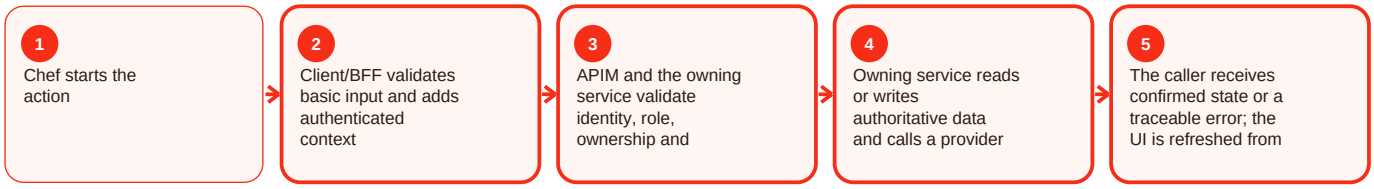
Summary

Chef verification is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef verification, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef verification - Operations, errors and deployment

Current implementation

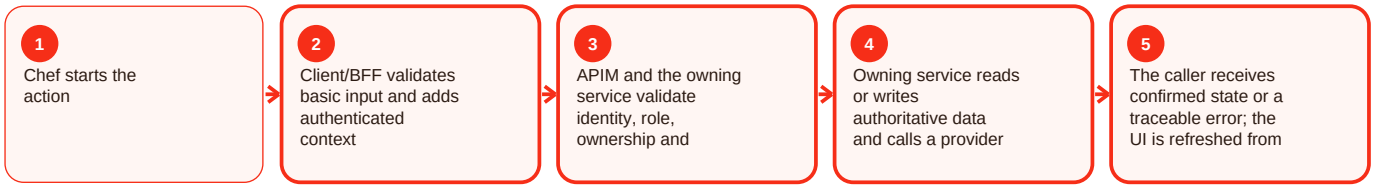
Summary

Chef verification is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef verification, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catalog admin - Overview and responsibility

Current implementation

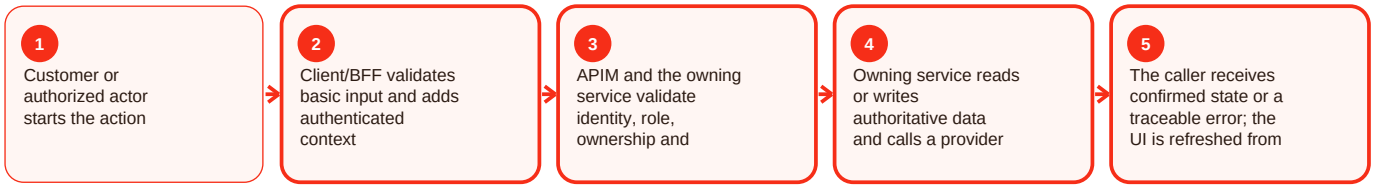
Summary

Catalog admin is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For catalog admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catalog admin - Functional behavior and data

Current implementation

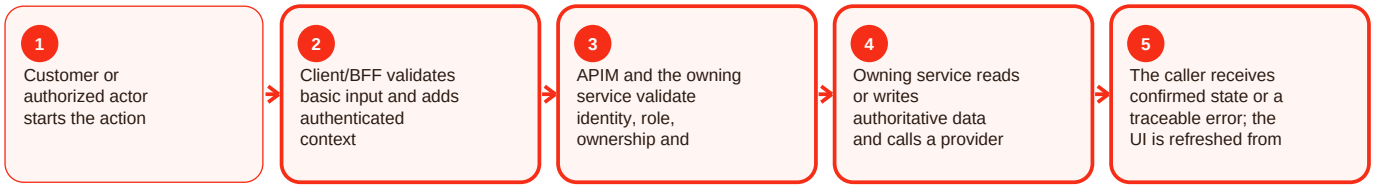
Summary

Catalog admin is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For catalog admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catalog admin - Operations, errors and deployment

Current implementation

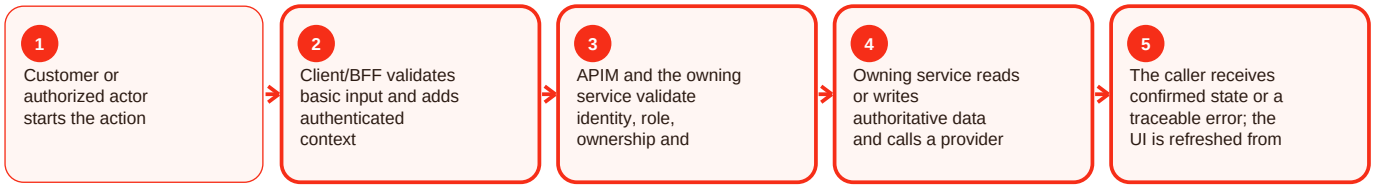
Summary

Catalog admin is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For catalog admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bulk pause / import - Overview and responsibility

Current implementation

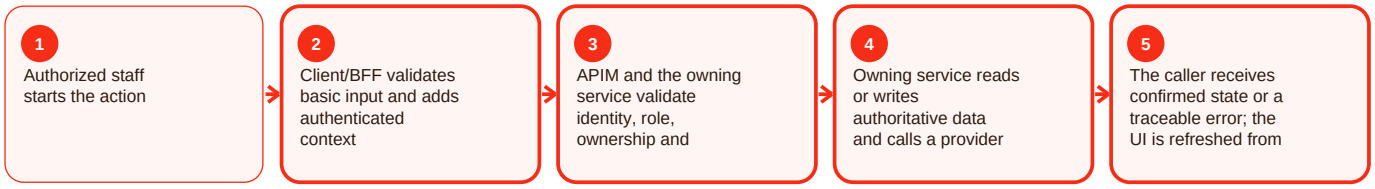
Summary

Bulk pause / import is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For bulk pause/import, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bulk pause / import - Functional behavior and data

Current implementation

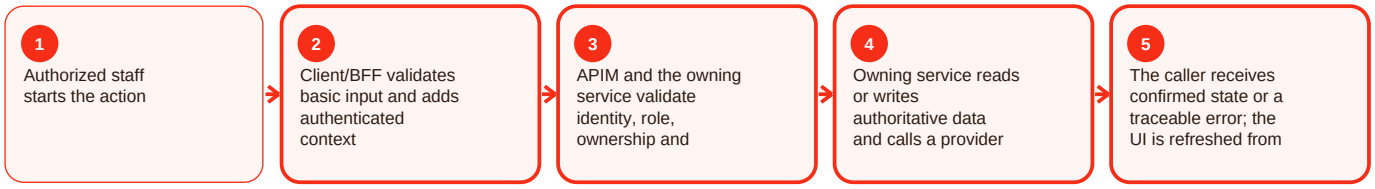
Summary

Bulk pause / import is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For bulk pause/import, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bulk pause / import - Operations, errors and deployment

Current implementation

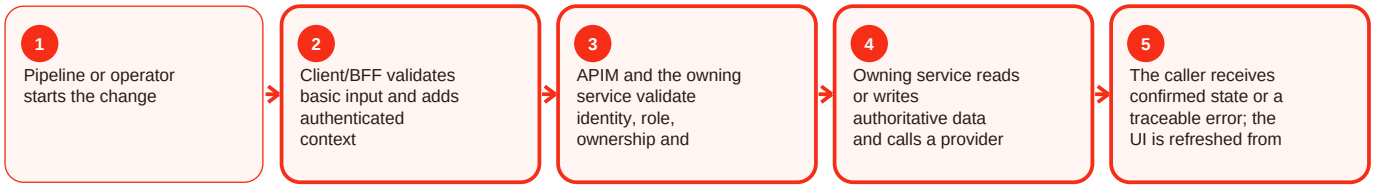
Summary

Bulk pause / import is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For bulk pause/import, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Order admin - Overview and responsibility

Current implementation

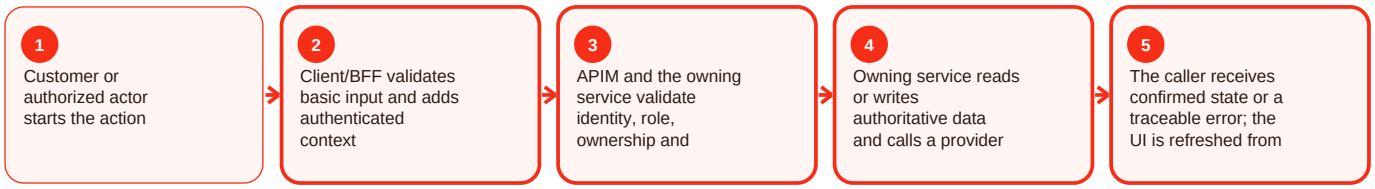
Summary

Order admin is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For order admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Order admin - Functional behavior and data

Current implementation

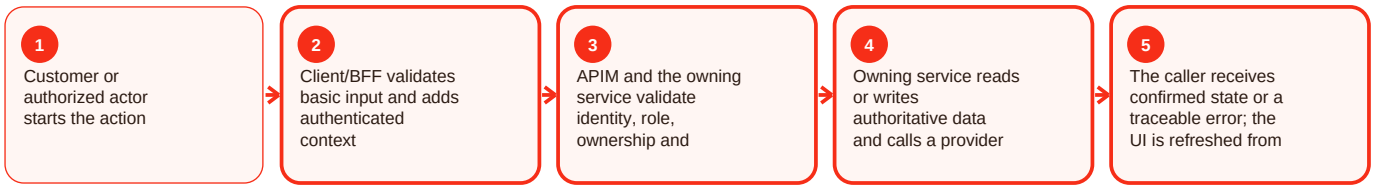
Summary

Order admin is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For order admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Order admin - Operations, errors and deployment

Current implementation

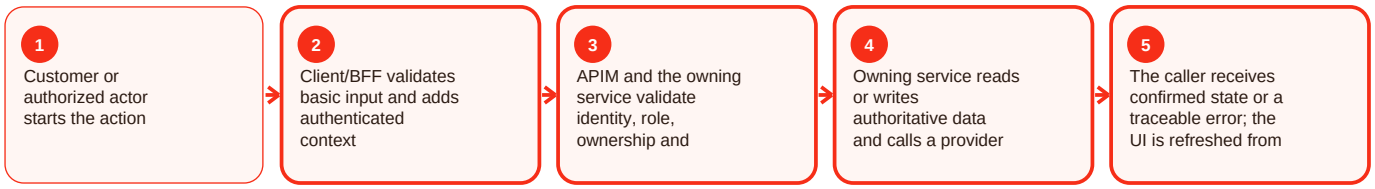
Summary

Order admin is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For order admin, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refund support - Overview and responsibility

Current implementation

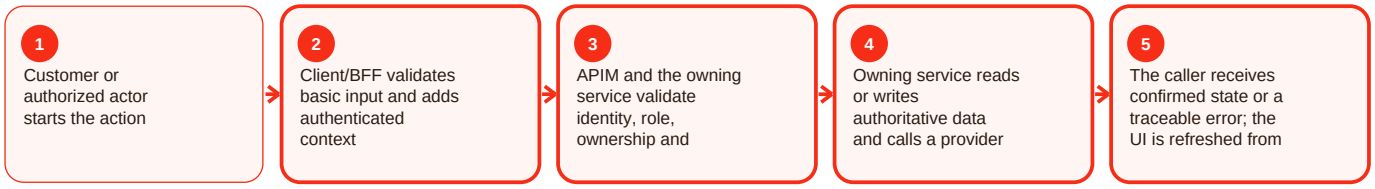
Summary

Refund support is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For refund support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refund support - Functional behavior and data

Current implementation

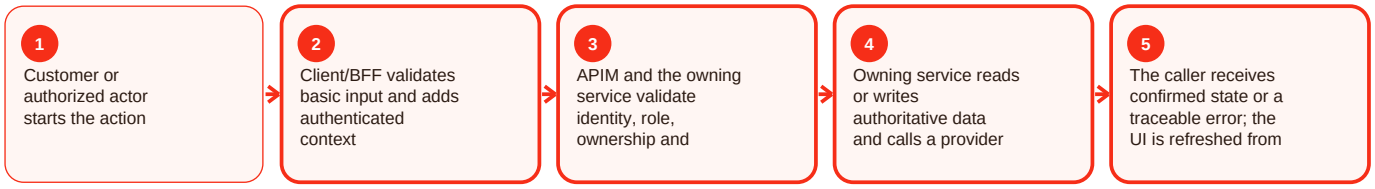
Summary

Refund support is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For refund support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refund support - Operations, errors and deployment

Current implementation

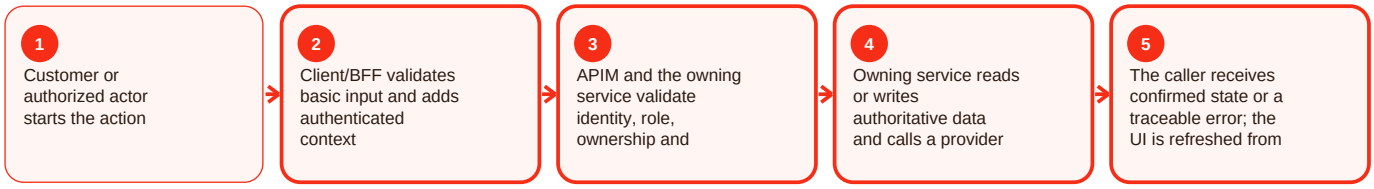
Summary

Refund support is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For refund support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Support operations - Overview and responsibility

Current implementation

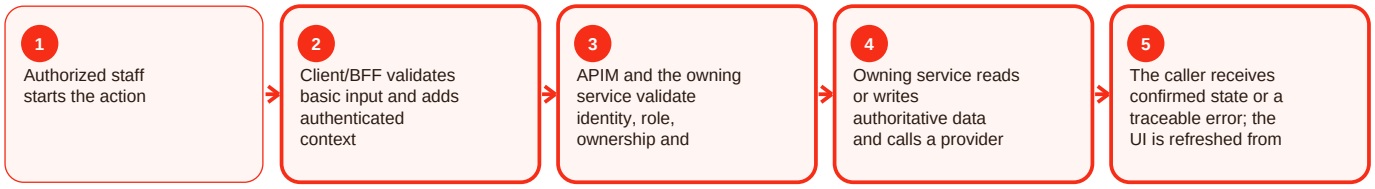
Summary

Support operations is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For support operations, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Support operations - Functional behavior and data

Current implementation

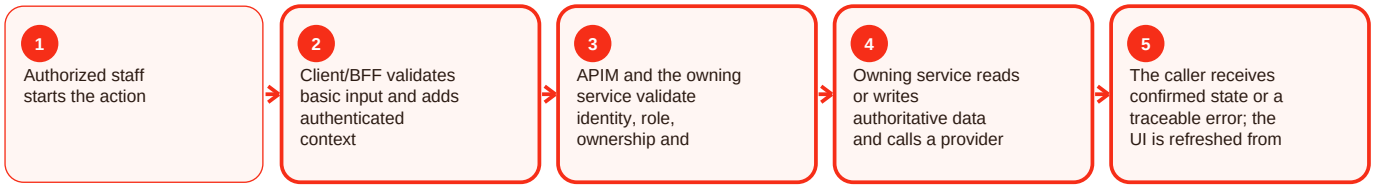
Summary

Support operations is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For support operations, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Support operations - Operations, errors and deployment

Current implementation

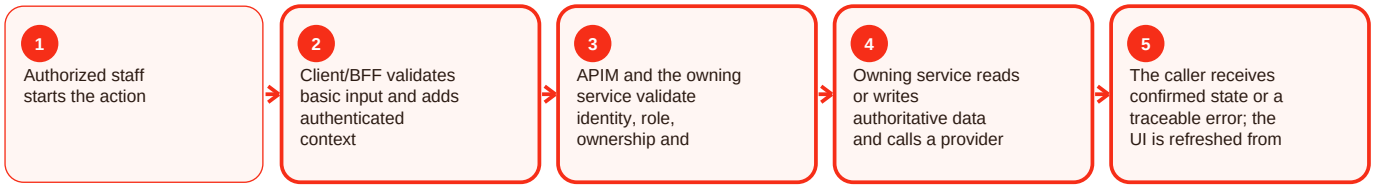
Summary

Support operations is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For support operations, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Staff access - Overview and responsibility

Current implementation

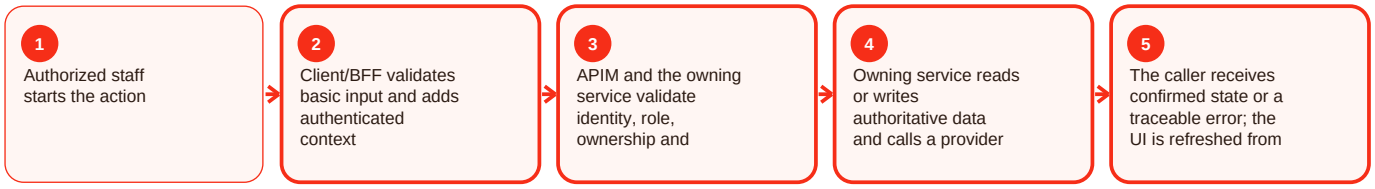
Summary

Staff access is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For staff access, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Staff access - Functional behavior and data

Current implementation

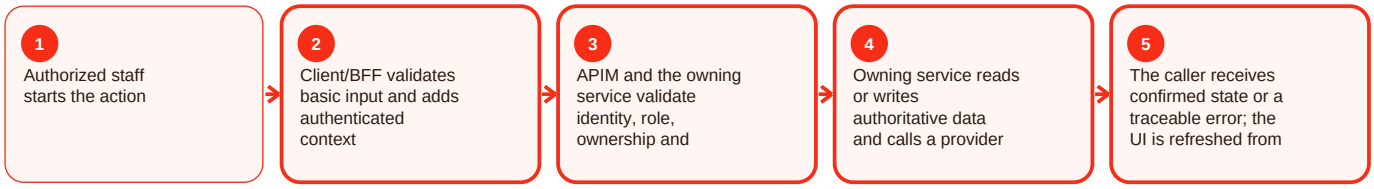
Summary

Staff access is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For staff access, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Staff access - Operations, errors and deployment

Current implementation

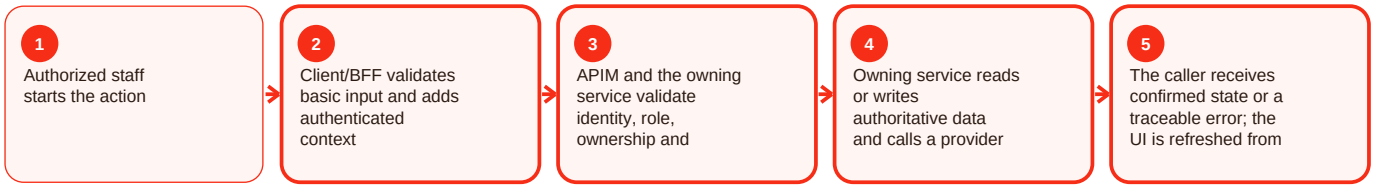
Summary

Staff access is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For staff access, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Promotions / community credit - Overview and responsibility

Current implementation

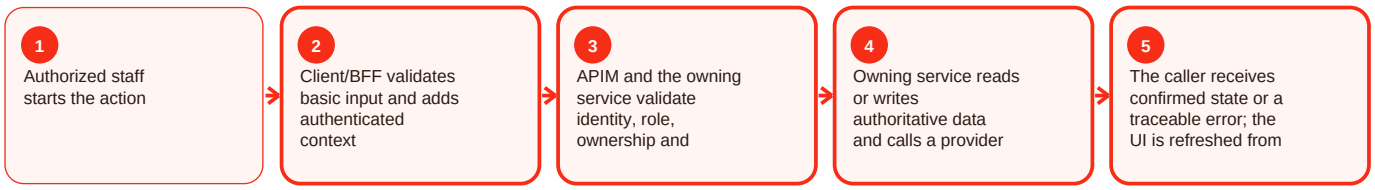
Summary

Promotions / community credit is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For promotions/community credit, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Promotions / community credit - Functional behavior and data

Current implementation

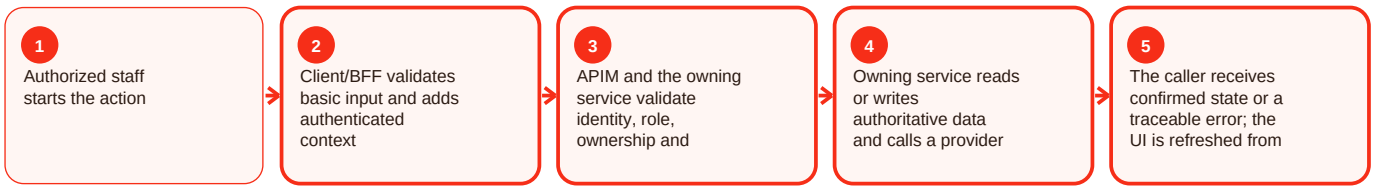
Summary

Promotions / community credit is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For promotions/community credit, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Promotions / community credit - Operations, errors and deployment

Current implementation

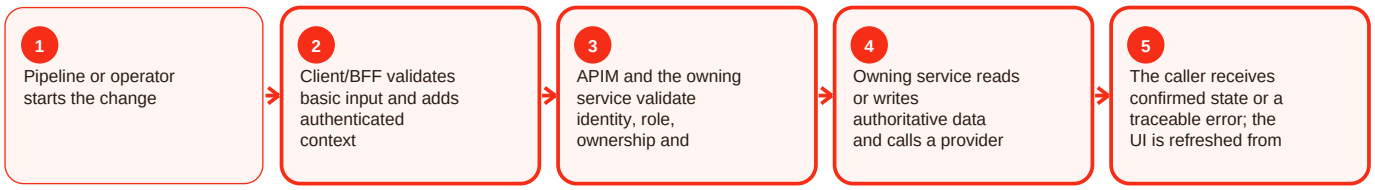
Summary

Promotions / community credit is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For promotions/community credit, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Consent / data rights support - Overview and responsibility

Current implementation

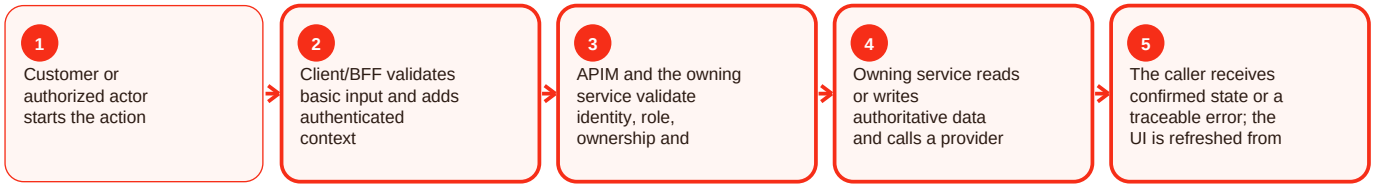
Summary

Consent / data rights support is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent/data rights support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Consent / data rights support - Functional behavior and data

Current implementation

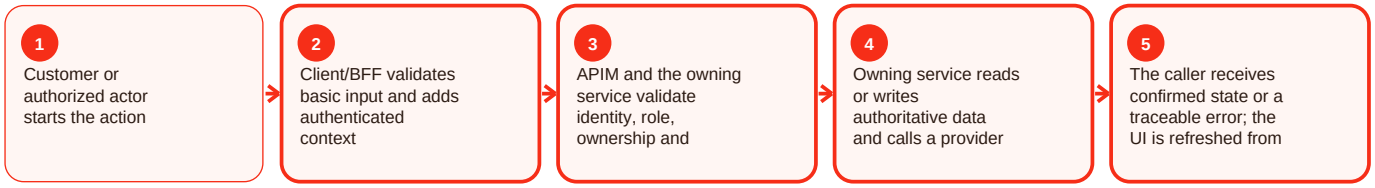
Summary

Consent / data rights support is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent/data rights support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Consent / data rights support - Operations, errors and deployment

Current implementation

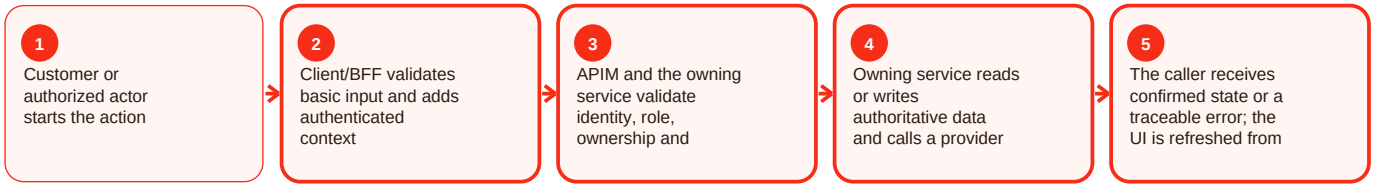
Summary

Consent / data rights support is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent/data rights support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catering support - Overview and responsibility

Current implementation

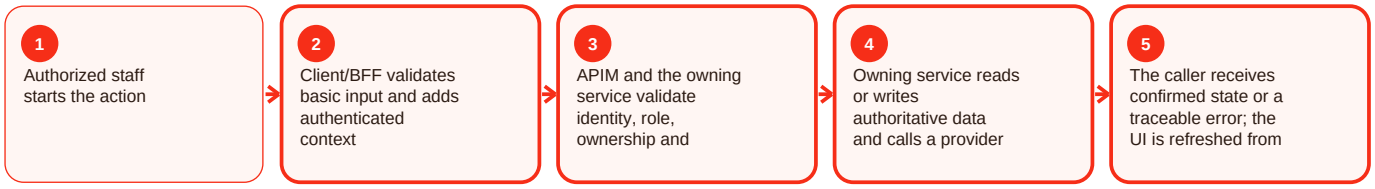
Summary

Catering support is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For catering support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catering support - Functional behavior and data

Current implementation

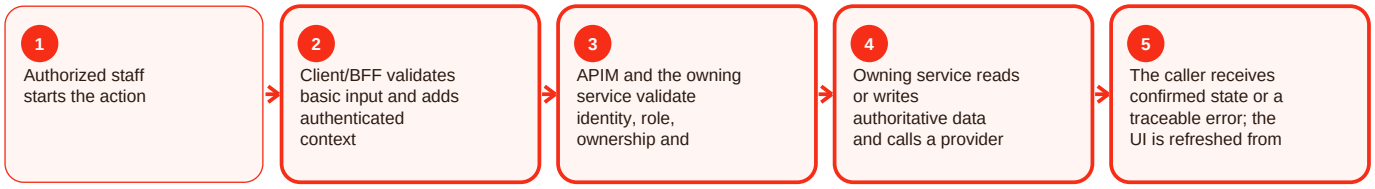
Summary

Catering support is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For catering support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catering support - Operations, errors and deployment

Current implementation

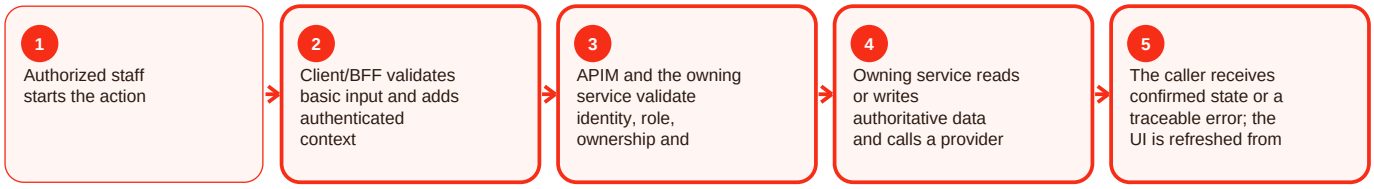
Summary

Catering support is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For catering support, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin status / settings / restart - Overview and responsibility

Current implementation

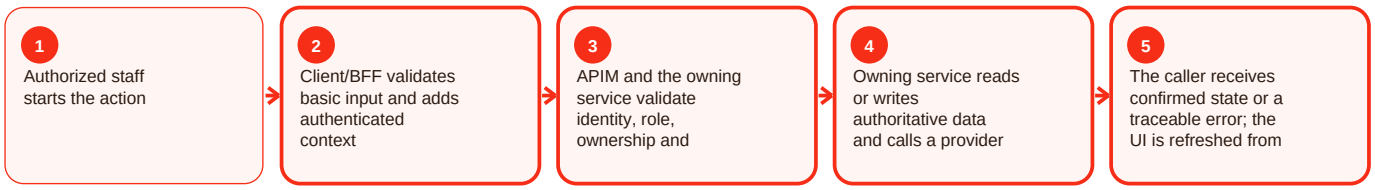
Summary

Admin status / settings / restart is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin status/settings/restart, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.
- Confirm least privilege and audit evidence for the sensitive action.

Repository gaps that must stay visible

- APIM README cites infra/apim/craves-openapi.yaml, but that literal artifact was not resolvable on main during this evidence snapshot.
- customer-web-next README names azure-pipelines-customer-web-next.yml, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.